第3章 关系数据库语言SQL

西安交通大学计算机科学与技术学院　　任雪斌

xuebinren@mail.xjtu.edu.cn

第3章 关系数据库语言SQL

3.1 SQL语言概述

3.2 SQL数据定义功能

3.3 SQL数据查询功能

3.4 SQL数据操纵功能

3.5 视图

3.6 SQL数据控制功能

3.7 嵌入式SQL

3.1 SQL语言概述

- 结构化查询语言SQL（Structured Query Language）是一种介于关系代数与关系演算之间的语言，其功能包括查询、操纵、定义和控制四个方面，是一个通用的的关系数据库语言
- SQL于1974年由Donald D. Chamberlin 和 Raymond F. Boyce(IBM)提出，并在IBM公司的System R上实现
- 目前，SQL已成为关系数据库的标准语言

3.1 SQL语言概述

➢SQL语言的版本

- 1986年，ANSI X3.135-1986，ISO/IEC 9075:1986，SQL-86
- 1989年，ANSI X3.135-1989，ISO/IEC 9075:1989，SQL-89 (SQL1) (About 156 pages)
- 1992年，ANSI X3.135-1992, ISO/IEC 9075:1992，SQL-92（SQL2）(>600 pages)
- 1999年，ISO/IEC 9075:1999, SQL:1999（SQL3）(1700 pages)
- 2003年，ISO/IEC 9075:2003, SQL:2003（SQL4）
- 2006年，ISO/IEC 9075-14:2006，SQL:2006
- SQL 2008(3777 pages)， SQL 2011，SQL 2016 ……

3.1 SQL语言概述

➢SQL的特点

1. 综合统一

- 集数据定义语言（DDL），数据查询语言（QL）数据操纵语言（DML），数据控制语言（DCL）功能于一体
- 可以独立完成数据库生命周期中的全部活动：
  - 定义关系模式，插入数据，建立数据库；
  - 对数据库中的数据进行查询和更新；
  - 数据库重构和维护
  - 数据库安全性、完整性控制等
- 用户数据库投入运行后，可根据需要随时逐步修改模式，不影响数据的运行
- 数据操作符统一

3.1 SQL语言概述

➢**SQL的特点**

2. 高度非过程化

- 非关系数据模型的数据操纵语言“面向过程”，必须指定存取路径
- SQL只要提出“做什么”，无须了解存取路径
- 存取路径的选择以及SQL的操作过程由系统自动完成

3.1 SQL语言概述

➢SQL的特点

3. 面向集合的操作方式

- 非关系数据模型采用面向记录的操作方式，操作对象是一条记录
- SQL采用集合操作方式
  - 操作对象、查找结果可以是元组的集合
  - 一次插入、删除、更新操作的对象可以是元组的集合

3.1 SQL语言概述

➢SQL的特点

4. 以同一种语法结构提供多种使用方式

- SQL是独立的语言

  能够独立地用于联机交互的使用方式

- SQL又是嵌入式语言

  SQL能够嵌入到高级语言（例如Python,C，C++，Java）程序中，供程序员设计程序时使用

3.1 SQL语言概述

➢SQL的特点

5. 语言简洁，易学易用

- SQL功能极强，完成核心功能只用了9个动词

表 3.1　SQL 语言的动词

SQL 功　能	动　　词
数 据 查 询	SELECT
数 据 定 义	CREATE，DROP，ALTER
数 据 操 纵	INSERT，UPDATE DELETE
数 据 控 制	GRANT，REVOKE

3.1 SQL语言概述

➢SQL的组成

核心SQL主要有四个部分:

(1) 数据定义语言，即SQL DDL，用于定义SQL模式、基本表、视图、索引等结构

(2) 数据操纵语言，即SQL DML，数据操纵分成数据查询和数据更新两类。其中数据更新又分成插入、删除和修改三种操作

(3) 数据控制语言，即SQL DCL，这一部分包括对基本表和视图的授权、完整性规则的描述、事务控制等内容。

(4) 嵌入式SQL语言的使用规定，这一部分内容涉及到SQL语句嵌入在宿主语言程序中的规则

3.2 SQL数据定义语言

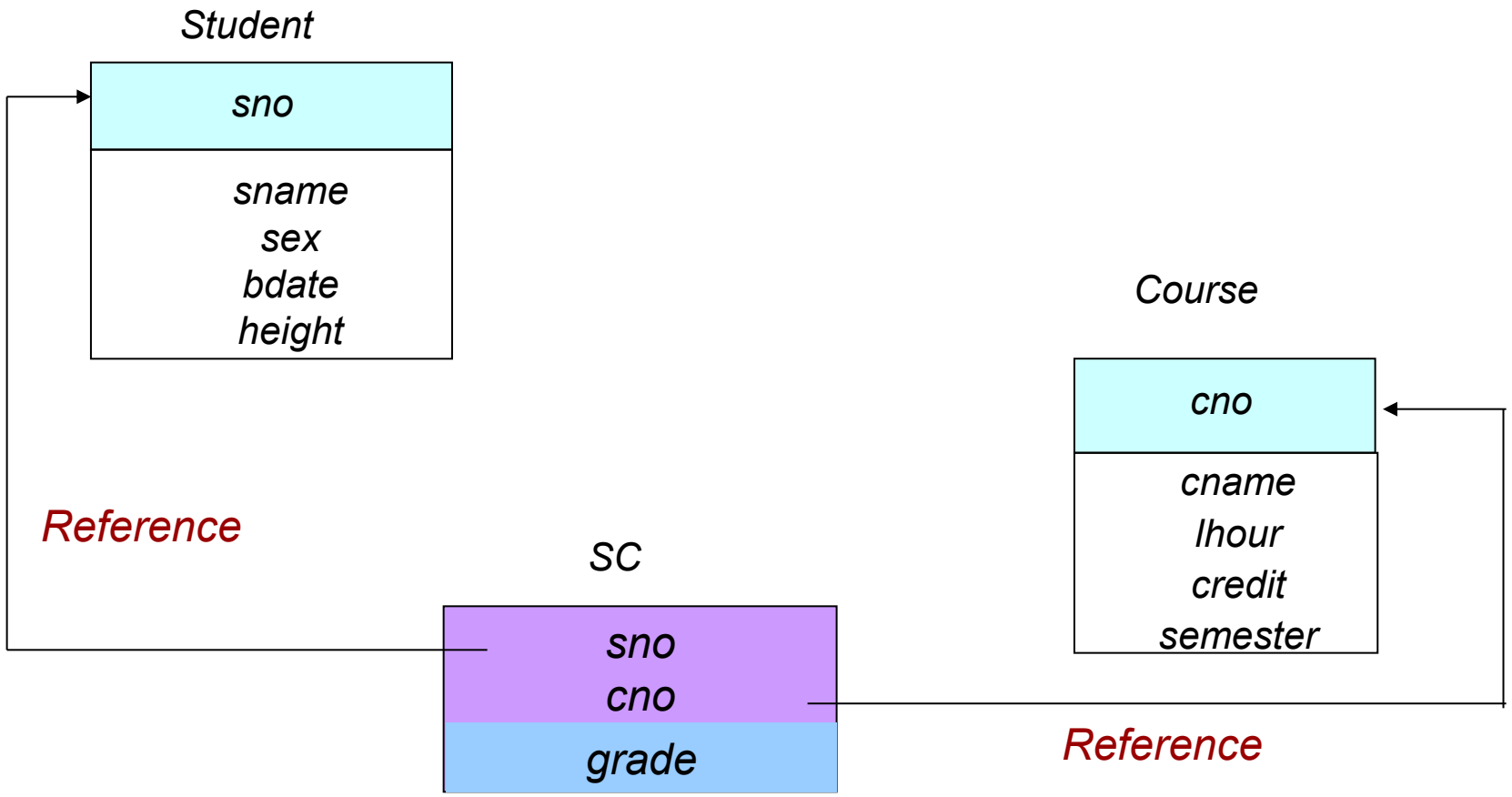
➢SQL的数据定义功能: 模式定义、表定义、视图和索引的定义

表 3.2　SQL 的数据定义语句

操作对象	操作方式		
	创　建	删　除	修　改
模式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视　图	CREATE VIEW	DROP VIEW	
索　引	CREATE INDEX	DROP INDEX	

3.2 SQL数据定义语言

➢创建基本表



3.2 SQL数据定义语言

➢创建基本表

例3-1：创建STUDENT,COURSE,SC三个基表

```
CREATE TABLE  STUDENT
     (SNO  CHAR(7)  NOT NULL,
      SNAME VARCHAR(10) NOT NULL,
      SEX  CHAR(1) NOT NULL,
      BDATE  DATE NOT NULL,
      HEIGHT  DEC(3,2) DEFAULT 0.0,
      PRIMARY KEY(SNO));  //定义主键，系统自动生成一个主键约束：student_pkey
```

```
CREATE TABLE COURSE
     (CNO  CHAR(6) NOT NULL,
      CNAME  VARCHAR(30) NOT NULL,
      LHOUR  SMALLINT NOT NULL,
      CREDIT DEC(1,0) NOT NULL,
      SEMESTER  CHAR(2) NOT NULL,
      PRIMARY KEY(CNO)); //定义主键， 系统自动生成一个主键约束：course_pkey
```

3.2 SQL数据定义语言

➢创建基本表

例3-1：创建STUDENT,COURSE,SC三个基表

```
CREATE TABLE  SC
      (SNO CHAR(7)  NOT NULL,
       CNO  CHAR(6) NOT NULL,
       GRADE  DEC(4,1) DEFAULT NULL,
       PRIMARY KEY(SNO,CNO),   //定义主键
       FOREIGN KEY(SNO)          //定义外键
          REFERENCES STUDENT
          ON DELETE CASCADE,
       FOREIGN KEY(CNO) //定义外键
          REFERENCES COURSE
          ON DELETE RESTRICT,
      CHECK ((GRADE IS NULL) OR (GRADE BETWEEN 0 AND 100))
);
```

3.2 SQL数据定义语言

➢创建基本表

CREATE TABLE <表名>(<列名> <数据类型> [完整性约束条件]

[,<列名> <数据类型> [完整性约束条件]]

……

[表级完整性约束条件])

3.2 SQL数据定义语言

➢SQL数据类型

- SQL中域的概念用数据类型来实现
- 定义表的属性时需要指明其数据类型及长度
- 选用哪种数据类型
  - 取值范围
  - 要做哪些运算

3.2 SQL数据定义语言

➢SQL数据类型

类型	数据类型举例	说明
binary	*binary large object(BLOB)*	以16进制格式存储二进制字符串的值。
boolean	*boolean*	存储真值 TRUE、FALSE或UNKNOWN
character	*char* *character varying(VARCHAR)* *national character(NCHAR)* *national character varying (NVARCHAR)* *character large object(CLOB)* *national character large object (NCLOB)*	可存储字符集中的任意字符组合。可变长度的数据类型允许字符长度变化，而其它数据类型字符长度是固定的。可变长度数据类型会自动删除后继的空格，定长数据类型会自动添加空格以补齐字符定义长度。
datalink	*datalink*	定义引用文件或其它非SQL环境的外部数据源。
interval	*interval*	指定一组时间值或者时间片段

3.2 SQL数据定义语言

➢SQL数据类型

类型	数据类型举例	说明
collection	*array* *multiset*	*array*是固定长度且有序的元素集合，*multiset*是可变长度，未排序的元素集合。元素必须是预定义数据类型。
numeric	*integer(INT)* *smallint* *bigint* *numeric* *decimal(DEC)* *float* *real* *double precision*	存储数字的准确值（整数或小数）或近似值（浮点型）。*INT*、*smallint*和*bigint*类型在预定义的精度和范围内存储数字的精确值。*numeric* 和*DEC*类型在可定义的精度和范围内存储数字的精确值。*float*是可定义精度的近似数类型。*real*和*double precision*是具有预定义精度的近似数类型。
temporal	*date* *time* *time with time zone* *timestamp* *timestamp with time zone*	这些数据类型处理关于时间的值。*date*和*time*分别处理日期和时间。有*with time zone*后缀的类型常包括一个时区的偏移。*timestamp*类型存储按照机器当前运行时间计算出来的值。
xml	*xml*	存储XML数据，可使用SQL数据类型的地方即可使用该类型。

3.2 SQL数据定义语言

➢创建基本表

- 列的完整性约束条件：
- 用SQL保留字NULL或NOT NULL指定当前列是否可以取空值或不允许取空值
- 用UNIQUE指定当前列取值不可重复
- 用DEFAULT <表达式>指定该列的缺省值或特别地用DEFAULT NULL指定当前列缺省取空值
- …

3.2 SQL数据定义语言

➢创建基本表

- 表级完整性约束的主要语法格式如下：

  PRIMARY KEY (<列名>) ,

  FOREIGN KEY　(<列名>) REFERENCES <表名>

  ON DELETE {RESTRICT | CASCADE | SET NULL},

  CHECK　　<逻辑条件表达式>

3.2 SQL数据定义语言

➢修改基本表

- 一般格式:

ALTER TABLE <表名>

ADD <新列名> <数据类型> [完整性约束]

DROP <完整性约束名>

DROP [COLUMN] <列名> [RESTRICT | CASCADE]

MODIFY<列名> <数据类型>

- 其中<表名>指定需要修改的基本表，ADD子句用于增加新列和新的完整性约束条件，DROP子句用于删除指定的完整性约束条件或列，MODIFY子句用于修改原有的列定义

3.2 SQL数据定义语言

➢修改基本表

- *例3-2：向STUDENT表增加“入学时间” (SCOME)列，其数据类型为日期型*

```
ALTER TABLE STUDENT ADD SCOME DATE DEFAULT NULL;
```

- *例3-3: 将学生姓名SNAME的长度增加到30*

```
ALTER TABLE STUDENT
MODIFY SNAME VARCHAR(30);
```

  - 修改原有的列定义有可能会破坏已有数据!

- *例3-4：删除关于学号为主键的约束*

```
ALTER TABLE STUDENT  DROP CONSTRAINT student_pkey;
```

3.2 SQL数据定义语言

➢删除基本表

- 一般格式：

  DROP TABLE <表名>

- *例3-5： 删除STUDENT表*

```
DROP TABLE STUDENT;
```

- 基本表定义一旦删除，表中的数据、在此表上建立的索引都将自动被删除掉，因此执行删除操作一定要格外小心

3.2 SQL数据定义语言

➢索引的概念

- RDBMS中索引一般采用B+树、HASH索引来实现
  - B+树索引具有动态平衡的优点
  - HASH索引具有查找速度快的特点
- 采用B+树，还是HASH索引 则由具体的RDBMS来决定
- 索引是关系数据库的内部实现技术，属于内模式的范畴
- CREATE INDEX语句定义索引时，可以定义索引是唯一索引、非唯一索引或聚簇索引

3.2 SQL数据定义语言

➢索引的建立与删除

- 建立索引的目的：加快查询速度
- 谁可以建立索引
  - DBA 或 表的属主（即建立表的人）
  - DBMS一般会自动建立以下列上的索引

    PRIMARY KEY

    UNIQUE
- 谁维护索引

  DBMS自动完成
- 使用索引

  DBMS自动选择是否使用索引以及使用哪些索引

3.2 SQL数据定义语言

➢索引的创建和删除

1. 创建索引

CREATE [UNIQUE] [CLUSTER] INDEX <索引名>

ON <表名> (<列名> [ASC | DESC] [,<列名> [ASC|DESC]]…)

例3-6：

```
CREATE INDEX SnoIndex ON SC(S# DESC)
CREATE UNIQUE INDEX SnoIndex ON SC(S# DESC, C#)
```

3.2 SQL数据定义语言

➢索引的创建和删除

2. 删除索引

DROP INDEX <索引名>

例3-7：

```
DROP INDEX CnameIndex
```

3.2 SQL数据定义语言

➢模式的创建和删除

- 创建模式实际上定义了一个命名空间
- 在这个空间中可以定义该模式包含的数据库对象，例如基本表、视图、索引等
- SQL模式由模式名标识，一般包括授权ID（通常会是系统中的一个用户名）和模式所含对象（如基本表、视图、域、约束和断言、触发器等）两方面内容的描述

3.2 SQL数据定义语言

➢模式的创建和删除

1. 创建模式

CREATE SCHEMA <模式名> AUTHORIZATION <所有者ID>

[创建基本表语句]

[创建视图语句]

[创建授权语句]

……

3.2 SQL数据定义语言

➢模式的创建和删除

- *例3-8:*

```
CREATE SCHEMA Dept AUTHORIZATION Zhang
CREATE TABLE Department ( id CHAR(6),
                          name   VARCHAR(10) )
```

3.2 SQL数据定义语言

➢模式的创建和删除

2. 删除模式

DROP SCHEMA <模式名> [RESTRICT | CASCADE]

- *例3-9:*

```
DROP SCHEMA dept CASCADE
```

3.2 SQL数据定义语言

➢模式与表

- 每一个基本表都属于某一个模式
- 一个模式包含多个基本表
- 定义基本表所属模式
  - 方法一：在表名中明显地给出模式名

    Create table “S-T”.Student（……）； /*模式名为 S-T*/

    Create table “S-T”.Course（……）；

    Create table “S-T”.SC（……）；
  - 方法二：在创建模式语句中同时创建表
  - 方法三：设置所属的模式

3.2 SQL数据定义语言

➢模式与表

- 创建基本表（其他数据库对象也一样）时，若没有指定模式，系统根据搜索路径来确定该对象所属的模式
- RDBMS会使用模式列表中第一个存在的模式作为数据库对象的模式名
- 若搜索路径中的模式名都不存在，系统将给出错误

3.3 SQL数据查询语言

- 基本查询语句
- 单表查询
- 连接查询
- 嵌套查询
- 集合查询

3.3 SQL数据查询语言

➢基本查询语句

- 一般格式

SELECT [ALL|DISTINCT]<目标列表达式>[,<目标列表达式>]...

FROM <表名或视图名>[,<表名或视图名>] ...

[WHERE <条件表达式>]

[GROUP BY <列名1>[HAVING <条件表达式>]]

[ORDER BY <列名2> [ASC|DESC]];

3.3 SQL数据查询语言

➢基本查询语句

- 整个SELECT语句的含义是，根据WHERE子句的条件表达式，从FROM子句指定的基本表或视图中找出满足条件的元组，再按SELECT子句中的目标列表达式，选出元组中的属性值形成结果表

- 如果有GROUP子句，则将结果按<列名1>的值进行分组，该属性列值相等的元组为一个组，每个组产生结果表中的一条记录,通常会在每组中作用集函数

- 如果GROUP子句带HAVING短语，则只有满足指定条件的组才予输出

- 如果有ORDER子句，则结果表还要按<列名2>的值的升序或降序进行排序

3.3 SQL数据查询语言

➢基本查询语句

- 一个典型的SQL查询采用如下的基本形式:

SELECT $A_1, A_2, \ldots, A_n$

FROM $r_1, r_2, \ldots, r_m$

WHERE P

- 其等价的关系代数表达式:

$$\Pi_{A_1,A_2,\ldots,A_n}(\sigma_P(r_1 \times r_2 \times \ldots \times r_m))$$

3.3 SQL数据查询语言

➢单表查询

- 查询仅涉及一个表:
  - 选择表中的若干列
  - 选择表中的若干元组
  - ORDER BY子句
  - 聚集函数
  - GROUP BY子句

3.3 SQL数据查询语言

➢单表表中若干列

- 查询指定列
- *例3-9：查询全体学生的学号与姓名*

```
SELECT Sno，Sname
FROM   Student;
```

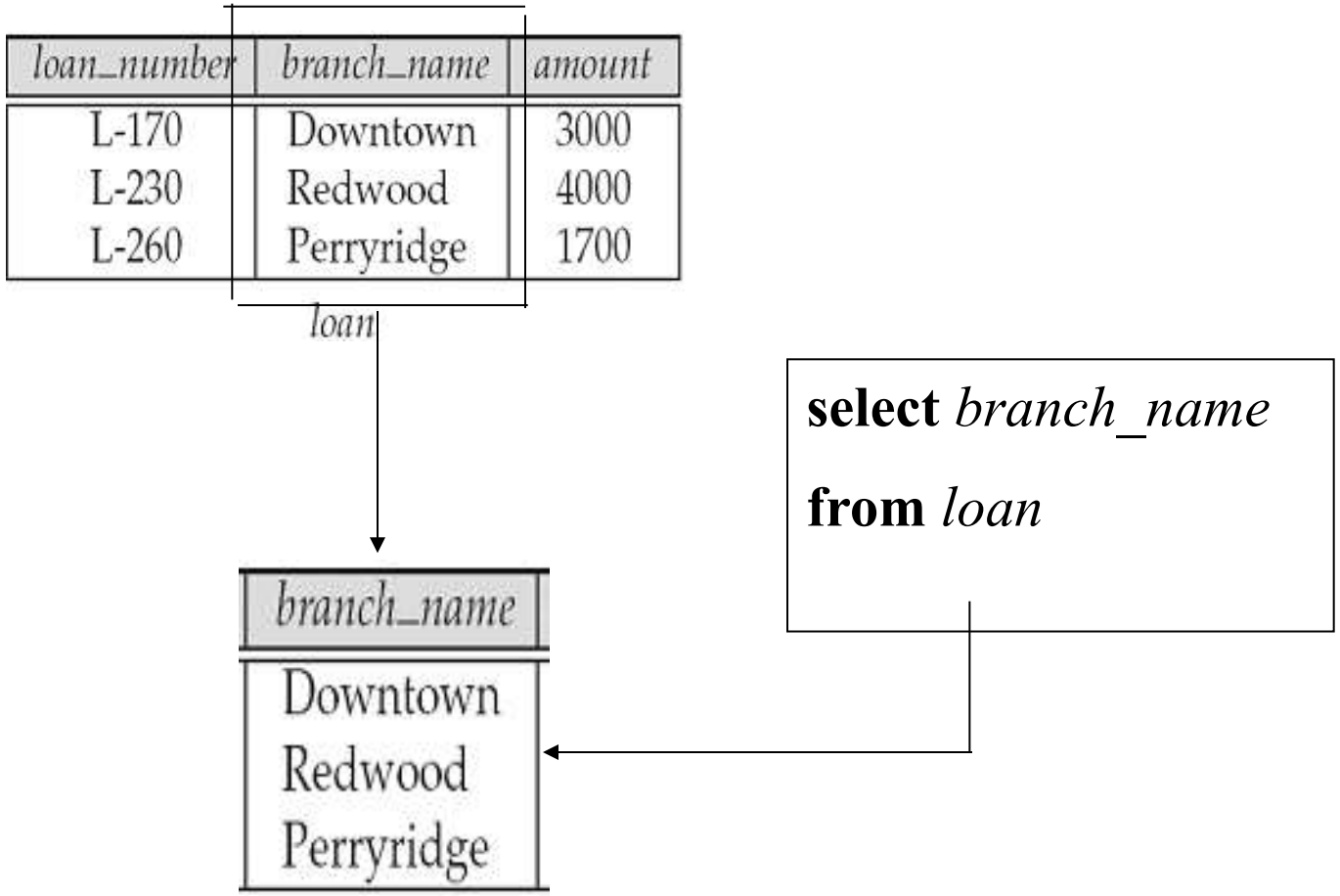
- *例3-10：查询全体学生的姓名、学号、所在系*

```
SELECT Sname，Sno，Sdept
FROM   Student;
```

3.3 SQL数据查询语言

➢选择表中若干列

```
select branch_name
from loan
```



3.3 SQL数据查询语言

➢查询全部列

- 选出所有属性列：
  - 在SELECT关键字后面列出所有列名
  - 将<目标列表达式>指定为 *
- *例3-10：查询全体学生的详细记录*

```
SELECT Sno，Sname，Ssex，Sage，Sdept
FROM   Student;
```

或

```
SELECT *
FROM   Student;
```

3.3 SQL数据查询语言

➢查询经过计算的值

- SELECT子句的<目标列表达式>可以为：
  - 算术表达式
  - 字符串常量
  - 函数
  - 列别名

3.3 SQL数据查询语言

➢查询经过计算的值

- *例3-11：查询全体学生的姓名及其当年年龄*

```
SELECT Sname，2026-Sage /*当年的年份为2025年*/
FROM   Student;
```

- *例3-12：查询全体学生的姓名、当年年龄*

```
SELECT Sname，  ‘Age at 2025:‘，2026-EXTRACT (year from Bdate)
FROM  Student;
```

3.3 SQL数据查询语言

➢查询经过计算的值

- 使用列别名改变查询结果的列标题:
- *例3-11：*

```
SELECT Sname，'Age at 2025:' Age_at_2025，2026-
EXTRACT (year from Bdate) Age FROM  Student;
```

3.3 SQL数据查询语言

➢查询表中的若干元组

- 消除取值重复的行

例3-14：查询选修了课程的学生学号(不取消重复行)

```
SELECT Sno   FROM SC;
```

等价于：

```
SELECT ALL  Sno  FROM SC;
```

- 如果指定DISTINCT关键词，则去掉表中重复的行

例3-15：查询选修了课程的学生学号(取消重复行)

```
SELECT DISTINCT Sno  FROM SC ;
```

- 注意:如果没有指定DISTINCT关键词，则缺省为ALL

3.3 SQL数据查询语言

➢选择表中的若干元组

loan_number	branch_name	amount
L-170	Downtown	3000
L-230	Redwood	4000
L-240	RedWood	5000
L260	Perryridge	1700

select distinct *branch_name*
from *loan*

The last result

branch_name
Downtown
Redwood
RedWood
Perryridge

branch_name
Downtown
Redwood
Perryridge

3.3 SQL数据查询语言

➢查询满足条件的元组

表3.4 常用的查询条件

查询条件	谓　词
比　较	=，>，<，>=，<=，!=，<>，!>，!<；NOT+上述比较运算符
确定范围	BETWEEN AND，NOT BETWEEN AND
确定集合	IN，NOT IN
字符匹配	LIKE，NOT LIKE
空　值	IS NULL，IS NOT NULL
多重条件（逻辑运算）	AND，OR，NOT

3.3 SQL数据查询语言

➢比较大小

- *例3-16：查询计算机系(CS)全体学生的名单*

```
SELECT Sname
FROM   Student
WHERE  Sdept='CS';
```

- *例3-17：查询所有年龄在20岁以下的学生姓名及其年龄*

```
SELECT Sname，Sage
FROM   Student
WHERE  Sage < 20;
```

- *例3-18：查询考试成绩有不及格的学生的学号*

```
SELECT DISTINCT Sno
FROM   SC
WHERE  Grade<60;
```

3.3 SQL数据查询语言

➢确定范围

- 谓词: BETWEEN … AND …
  NOT BETWEEN … AND …
- *例3-19：查询年龄在20~23岁（包括20岁和23岁）之间的学生的姓名、系别和年龄*

```
SELECT Sname，Sdept，Sage
FROM   Student
WHERE  Sage BETWEEN 20 AND 23；
```

- *例3-20：查询年龄不在20~23岁之间的学生姓名、系别和年龄*

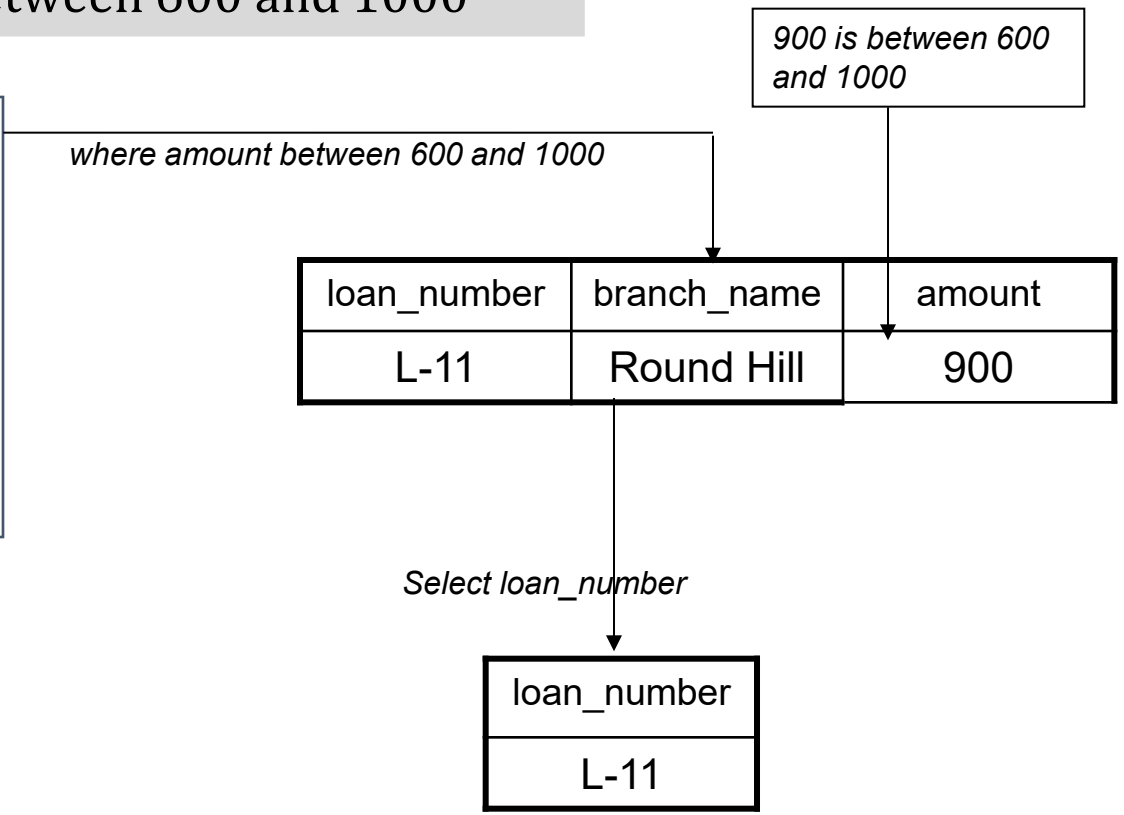
```
SELECT Sname，Sdept，Sage
FROM   Student
WHERE  Sage NOT BETWEEN 20 AND 23；
```

3.3 SQL数据查询语言

➢确定范围

```
select loan_number
        from loan
        where amount between 600 and 1000
```

loan_number	branch_name	amount
L-11	Round Hill	900
L-14	Downtown	1500
L-15	Perryridge	1500
L-16	Perryridge	1300
L-17	Downtown	1000
L-23	Redwood	2000
L-93	Mianus	500



3.3 SQL数据查询语言

➢确定集合

- 谓词：IN <值表>，NOT IN <值表>
- *例3-21：查询信息系（IS）、数学系（MA）和计算机科学系（CS）学生的姓名和性别*

```
SELECT Sname，Ssex
FROM  Student
WHERE Sdept IN ( 'IS'，'MA'，'CS' );
```

- *例3-22：查询既不是信息系、数学系，也不是计算机科学系的学生的姓名和性别*

```
SELECT Sname，Ssex
FROM  Student
WHERE Sdept NOT IN ( 'IS'，'MA'，'CS' );
```

3.3 SQL数据查询语言

➢字符匹配

- 谓词：[NOT] LIKE ‘<匹配串>’ [ESCAPE ‘ <换码字符>’]

1）匹配串为固定字符串

- *例3-22：查询学号为201215121的学生的详细情况*

```
SELECT *
FROM   Student
WHERE  Sno LIKE ‘201215121‘;
```

等价于：

```
SELECT *
FROM   Student
WHERE  Sno = ‘201215121‘;
```

3.3 SQL数据查询语言

➢字符匹配

2）匹配串为含通配符的字符串

- *例3-23：查询所有姓刘学生的姓名、学号和性别*

```
SELECT Sname，Sno，Ssex
FROM  Student
WHERE  Sname LIKE '刘%';
```

- *例3-24：查询姓"欧阳"且全名为三个字的学生的姓名*

```
SELECT Sname
FROM  Student
WHERE  Sname LIKE '欧阳_' ;
```

3.3 SQL数据查询语言

➢字符匹配

- *例3-25：查询名字中第2个字为"阳"字的学生的姓名和学号*

```
SELECT Sname
FROM   Student
WHERE  Sname LIKE '_阳%' ;
```

- *例3-26：查询所有不姓刘的学生姓名*

```
SELECT Sname，Sno，Ssex
FROM   Student
WHERE  Sname NOT LIKE '刘%';
```

3.3 SQL数据查询语言

➢涉及空值的查询

- 谓词： IS NULL 或 IS NOT NULL
- *例3-29：某些学生选修课程后没有参加考试，所以有选课记录，但没有考试成绩。查询缺少成绩的学生的学号和相应的课程号*

```
SELECT Sno，Cno
FROM  SC
WHERE  Grade IS NULL
```

- *例3-30：查所有有成绩的学生学号和课程号*

```
SELECT Sno，Cno
FROM  SC
WHERE  Grade IS  NOT NULL
```

- 注意:不能用 “=” 代替 “IS”

3.3 SQL数据查询语言

➢多重条件查询

- 逻辑运算符：AND和 OR来联结多个查询条件
  - AND的优先级高于OR
  - 可以用括号改变优先级
- 可用来实现多种其他谓词
  - [NOT] IN
  - [NOT] BETWEEN … AND …

3.3 SQL数据查询语言

➢多重条件查询

- *例3-31：查询计算机系年龄在20岁以下的学生姓名*

```
SELECT Sname
FROM   Student
WHERE  Sdept= 'CS' AND Sage<20;
```

3.3 SQL数据查询语言

➢多重条件查询

```
select loan_number
from loan
where branch_name = 'Perryridge' and amount > 1400
```

loan_number	branch_name	amount
L-11	Round Hill	900
L-14	Downtown	1500
L-15	Perryridge	1500
L-16	Perryridge	1300
L-17	Downtown	1000
L-23	Redwood	2000
L-93	Mianus	500

where *branch_name* = 'Perryridge'

loan_number	branch_name	amount
L-15	Perryridge	1500
L-16	Perryridge	1300

amount > 1400

loan_number	branch_name	amount
L-15	Perryridge	1500

Select loan_number

loan_number
L-15

3.3 SQL数据查询语言

➢ORDER BY子句

- ORDER BY子句
  - 可以按一个或多个属性列排序
  - 升序：ASC；降序：DESC；缺省值为升序
- 当排序列含空值时
  - ASC：排序列为空值的元组最先显示
  - DESC：排序列为空值的元组最后显示

3.3 SQL数据查询语言

➢ORDER BY子句

- *例3-32：查询选修了CS-03课程的学生的学号及其成绩，查询结果按分数降序排列*

```
SELECT Sno，Grade
FROM   SC
WHERE  Cno= 'CS-03'
ORDER BY Grade DESC;
```

- *例3-33：查询全体学生情况，查询结果按所在系的系号升序排列，同一系中的学生按年龄降序排列*

```
SELECT *
FROM   STUDENT
ORDER BY Sdept ASC，Sage DESC;
```

3.3 SQL数据查询语言

➢SQL函数

- SQL函数主要包括标量函数和聚集函数两类
- 标量函数的运算对象是一个记录行在某个属性上的具体取值，大致可以分为字符、数学、时间、类型转换等几种形式。例如，函数LENGTH()可以计算一个字符串类型数值的长度，ABS()可以计算一个数值类型的绝对值

3.3 SQL数据查询语言

➢聚集函数

- 聚集函数是SQL中具有统计性质的一类函数，其运算对象通常是记录的集合或一组记录在某个列上的全部取值，聚集函数的返回结果一般将会是惟一的一个确定值

3.3 SQL数据查询语言

➢聚集函数

- 计数

  COUNT（[DISTINCT|ALL] *）

  COUNT（[DISTINCT|ALL] <列名>）

- 计算总和

  SUM（[DISTINCT|ALL] <列名>）

- 计算平均值

  AVG（[DISTINCT|ALL] <列名>）

- 最大最小值

  MAX（[DISTINCT|ALL] <列名>）

  MIN（[DISTINCT|ALL] <列名>）

3.3 SQL数据查询语言

➢聚集函数

- *例3-34：查询学生总人数*

```
SELECT COUNT(*)
FROM   Student;
```

- *例3-35：查询选修了课程的学生人数*

```
SELECT COUNT(DISTINCT Sno)
FROM   SC;
```

- *例3-36：计算‘CS-01’课程的学生平均成绩*

```
SELECT AVG(Grade)
FROM   SC
WHERE  Cno='CS-01';
```

3.3 SQL数据查询语言

- *例3-37：查询选修‘CS-01’课程的学生最高分数*

```
SELECT MAX(Grade)
FROM   SC
WHERE  Cno='CS-01';
```

- *例3-38：查询学生‘200215012’选修课程的总学分数*

```
SELECT SUM(Ccredit)
FROM   SC，Course
WHERE Sno='200215012' AND SC.Cno=Course.Cno;
```

3.3 SQL数据查询语言

➢GROUP BY子句

- GROUP BY子句分组：
  - 细化聚集函数的作用对象
  - 未对查询结果分组，聚集函数将作用于整个查询结果
  - 对查询结果分组后，聚集函数将分别作用于每个组
  - 作用对象是查询的中间结果表
  - 按指定的一列或多列值分组，值相等的为一组

3.3 SQL数据查询语言

➢GROUP BY子句

- *例3-39：求各个课程号及相应的选课人数*

```
SELECT Cno，COUNT(Sno)
FROM  SC
GROUP BY Cno;
```

Q：平均成绩最高的学生学号及成绩

```
SELECT S#, MAX(AVG(Grade))
FROM  SC
GROUP BY S#
```

sno	cno	grade
801	01	92
801	02	78
801	03	85
802	01	82
802	02	90
803	01	88

3.3 SQL数据查询语言

➢GROUP BY子句

列出每个学生的平均成绩

	SNO	CNO	G
90	s1	c1	84
	s1	c2	90
	s1	c3	96
85	s2	c1	80
	s2	c2	90
92	s3	c2	96
	s3	c3	88

group by SNO

列出每门课程的平均成绩

SNO	CNO	G	
s1	c1	84	90
s1	c2	90	
s1	c3	96	92
s2	c1	80	
s2	c2	90	
s3	c2	96	92
s3	c3	88	

group by CNO

3.3 SQL数据查询语言

➢HAVING短语

- HAVING短语与WHERE子句的区别：
  - 作用对象不同
  - WHERE子句作用于基表或视图，从中选择满足条件的元组
  - HAVING短语作用于组，从中选择满足条件的组

3.3 SQL数据查询语言

➢HAVING短语

- *例3-40：查询选修了3门以上课程的学生学号*

```
SELECT Sno
FROM   SC
GROUP BY Sno
HAVING  COUNT(*) >= 3;
```

3.3 SQL数据查询语言

loan_number	branch_name	amount
L-11	Round Hill	900
L-14	Downtown	1500
L-15	Perryridge	1500
L-16	Perryridge	1300
L-17	Downtown	1000
L-23	Redwood	2000
L-93	Mianus	500

Group by branch_name

loan_number	branch_name	amount
L-11	Round Hill	900
L-14	Downtown	1500
L-17	Downtown	1000
L-15	Perryridge	1500
L-16	Perryridge	1300
L-23	Redwood	2000
L-93	Mianus	500

avg (amount)

branch_name	amount
Round Hill	900
Downtown	1250
Perryridge	1400
Redwood	2000
Mianus	500

Having avg (amount) >1200

branch_name	amount
Downtown	1250
Perryridge	1400
Redwood	2000

3.3 SQL数据查询语言

➢连接查询（多表查询）

- 连接查询：同时涉及多个表的查询
- 连接条件或连接谓词：用来连接两个表的条件
- 一般格式：
  - [<表名1>.]<列名1> <比较运算符> [<表名2>.]<列名2>
  - [<表名1>.]<列名1> BETWEEN [<表名2>.]<列名2> AND [<表名2>.]<列名3>
  - 连接字段：连接谓词中的列名称
  - 连接条件中的各连接字段类型必须是可比的，但名字不必是相同的

3.3 SQL数据查询语言

➢连接查询

- 等值与非等值连接查询
- 自连接
- 外连接
- 复合条件连接

3.3 SQL数据查询语言

➢等值与非等值连接查询

- 等值连接：连接运算符为=
- *例3-41：查询每个学生及其选修课程的情况*

```
SELECT  Student.*，SC.*  (*)
FROM    Student，SC
WHERE   Student.Sno = SC.Sno；
```

3.3 SQL数据查询语言

➢等值与非等值连接查询

- 自然连接(?×):
- *例3-42：对[例3-41]用自然连接完成*

```
SELECT Student.Sno，Sname，Ssex，Sage，Sdept，Cno，Grade
FROM  Student，SC
WHERE Student.Sno = SC.Sno;
```

3.3 SQL数据查询语言

➢等值与非等值连接查询

- 自连接：一个表与其自己进行连接
- 需要给表起别名以示区别
- 由于所有属性名都是同名属性，因此必须使用别名前缀
- *例3-43：查询每一门课的间接先修课（即先修课的先修课）*

```
SELECT FIRST.Cno，SECOND.Cpno
FROM Course FIRST，Course SECOND
WHERE FIRST.Cpno = SECOND.Cno;
```

3.3 SQL数据查询语言

➢自连接

FIRST表（Course表）

Cno	Cname	Cpno	Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

3.3 SQL数据查询语言

➢自连接

SECOND表（Course表）

Cno	Cname	Cpno	Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

3.3 SQL数据查询语言

➢自连接

查询结果：

FIRST

Cno	Cname	Cpno	Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

SECOND

Cno	Cname	Cpno	Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

Cno	Pcno
1	7
3	5
4	null
5	6
7	null

3.3 SQL数据查询语言

➢外连接

- 外连接与普通连接的区别
  - 普通连接操作只输出满足连接条件的元组
  - 外连接操作以指定表为连接主体，将主体表中不满足连接条件的元组一并输出
- *例3-44：改写[例3-42]*

```
SELECT Student.Sno，Sname，Ssex，Sage，Sdept，Cno，Grade
FROM  Student  LEFT OUTER JOIN SC ON (Student.Sno=SC.Sno)；
```

3.3 SQL数据查询语言

➢外连接

- 左外连接
  - 列出左边关系（如本例Student）中所有的元组
- 右外连接
  - 列出右边关系中所有的元组

3.3 SQL数据查询语言

➢复合条件连接

- 复合条件连接：WHERE子句中含多个连接条件
- *例3-44：查询选修2号课程且成绩在90分以上的所有学生*

```
SELECT Student.Sno, Sname
 FROM   Student, SC
WHERE  Student.Sno = SC.Sno AND /* 连接谓词*/
SC.Cno= '2' AND SC.Grade > 90;
/* 其他限定条件 */
```

3.3 SQL数据查询语言

➢复合条件连接

- *例3-45：查询每个学生的学号、姓名、选修的课程名及成绩*

```
SELECT Student.Sno，Sname，Cname，Grade
FROM  Student，SC，Course  /*多表连接*/
WHERE  Student.Sno = SC.Sno
       AND SC.Cno = Course.Cno;
```

3.3 SQL数据查询语言

➢嵌套查询

- 嵌套查询概述
  - 一个SELECT-FROM-WHERE语句称为一个查询块
  - 将一个查询块嵌套在另一个查询块的WHERE子句或HAVING短语的条件中的查询称为嵌套查询

3.3 SQL数据查询语言

➢嵌套查询

- *例3-46：选修了2号课程的学生姓名*

```
SELECT Sname                    /*外层查询/父查询*/
FROM Student
WHERE Sno IN（SELECT Sno   /*内层查询/子查询*/
        FROM SC
        WHERE Cno='2'）;
```

3.3 SQL数据查询语言

➢嵌套查询

- *例3-46：选修了2号课程的学生姓名*

```
SELECT Sname                    /*外层查询/父查询*/
FROM Student
WHERE '2 ' IN（SELECT Cno   /*内层查询/子查询*/
        FROM SC
        WHERE Student.Sno=SC.Sno）;
```

3.3 SQL数据查询语言

➢带有比较运算符的子查询

- 当能确切知道内层查询返回单值时，可用比较运算符（>，<，=，>=，<=，!=或< >)
- 与SOME、ANY或ALL谓词配合使用

3.3 SQL数据查询语言

➢带有比较运算符的子查询

- *例3-50：假设一个学生只可能在一个系学习，并且必须属于一个系，则在[例3-45]可以用 = 代替IN：*

```
SELECT Sno，Sname，Sdept
FROM   Student
WHERE Sdept  =
       (SELECT Sdept
       FROM   Student
      WHERE Sname= '刘晨' );
```

?

3.3 SQL数据查询语言

➢带有比较运算符的子查询

- 子查询一定要跟在比较符之后,下面是一个错误的例子:

```
SELECT Sno，Sname，Sdept
FROM   Student
WHERE ( SELECT Sdept
        FROM Student
        WHERE Sname=‘刘晨’ )
       = Sdept;
```

?

3.3 SQL数据查询语言

➢带有比较运算符的子查询

- *例3-51：找出每个学生超过他已选修课程平均成绩的课程号*

```
SELECT Sno，  Cno
FROM  SC  X
WHERE Grade >=(SELECT AVG(Grade)
        FROM  SC Y
        WHERE Y.Sno=X.Sno);
```

相关子查询

3.3 SQL数据查询语言

➢带有比较运算符的子查询

- 执行过程：

1. 从外层查询中取出SC的一个元组x，将元组x的Sno值（如’200215121’）传送给内层查询。

```
SELECT AVG(Grade)
FROM SC Y
WHERE Y.Sno='200215121';
```

2. 执行内层查询，得到平均值（如85），
用该值代替内层查询，得到外层查询：

```
SELECT Sno， Cno
FROM SC X
WHERE Grade >=85；
```

3.3 SQL数据查询语言

➢带有比较运算符的子查询

- 执行过程:

3. 执行这个查询，得到

('200215121'，'CS-01')

('200215121'，'CS-03')

4.外层查询取出下一个元组重复做上述1至3步骤，直到外层的SC元组全部处理完毕。结果为:

('200215121'，'CS-01')

('200215121'，'CS-03')

('200215122'，'CS-02')

3.3 SQL数据查询语言

➢带有SOME（ANY）或ALL谓词的子查询

- 谓词语义
  - SOME(ANY)：任意一个值
  - ALL：所有值

3.3 SQL数据查询语言

➢带有SOME（ANY）或ALL谓词的子查询

- 需要配合使用比较运算符

> SOME	大于子查询结果中的某个值
> ALL	大于子查询结果中的所有值
< SOME	小于子查询结果中的某个值
< ALL	小于子查询结果中的所有值
>= SOME	大于等于子查询结果中的某个值
>= ALL	大于等于子查询结果中的所有值
<= SOME	小于等于子查询结果中的某个值
<= ALL	小于等于子查询结果中的所有值
= SOME	等于子查询结果中的某个值
=ALL	等于子查询结果中的所有值（通常没有实际意义）
!=（或<>） SOME	不等于子查询结果中的某个值
!=（或<>）ALL	不等于子查询结果中的任何一个值

3.3 SQL数据查询语言

➢带有SOME（ANY）或ALL谓词的子查询

- *例3-52：查询其他系中比计算机科学系某一学生年龄小的学生姓名和年龄*

```
SELECT Sname，Sage
FROM   Student
WHERE Sage < SOME (SELECT Sage
        FROM    Student
        WHERE Sdept= ' CS ‘)
AND Sdept <> ‘CS' ; /*父查询块中的条件 */
```

3.3 SQL数据查询语言

➢带有ANY（SOME）或ALL谓词的子查询

- 执行过程：

1. RDBMS执行此查询时，首先处理子查询，找出CS系中所有学生的年龄，构成一个集合(20，19)
2. 处理父查询，找所有不是CS系且年龄小于20或19的学生

3.3 SQL数据查询语言

➢带有ANY（SOME）或ALL谓词的子查询

- *例3-53：查询其他系中比计算机科学系所有学生年龄都小的学生姓名及年龄。*
- 方法一：用ALL谓词

```
SELECT Sname，Sage
FROM   Student
WHERE Sage < ALL (SELECT Sage
        FROM    Student
        WHERE Sdept= ' CS ‘)
AND Sdept <> ‘CS' ;
```

3.3 SQL数据查询语言

➢带有ANY（SOME）或ALL谓词的子查询

- 例3-53：查询其他系中比计算机科学系所有学生年龄都小的学生姓名及年龄。
- 方法二：用聚集函数

```
SELECT Sname，Sage
FROM   Student
WHERE Sage < (SELECT MIN(Sage)
        FROM    Student
        WHERE Sdept= ' CS ‘)
AND Sdept <> ‘CS' ;
```

3.3 SQL数据查询语言

➢带有ANY（SOME）或ALL谓词的子查询

表3.5 SOME（或ANY），ALL谓词与聚集函数、IN谓词的等价转换关系

	=	<>或!=	<	<=	>	>=
SOME	IN	--	<MAX	<=MAX	>MIN	>= MIN
ALL	--	NOT IN	<MIN	<= MIN	>MAX	>= MAX

3.3 SQL数据查询语言

➢带有EXISTS谓词的子查询

- EXISTS谓词
  - 存在量词 ∃
  - 带有EXISTS谓词的子查询不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”
    - 若内层查询结果非空，则外层的WHERE子句返回真值
    - 若内层查询结果为空，则外层的WHERE子句返回假值
  - 由EXISTS引出的子查询，其目标列表达式通常都用* ，因为带EXISTS的子查询只返回真值或假值，给出列名无实际意义
- NOT EXISTS谓词
  - 若内层查询结果非空，则外层的WHERE子句返回假值
  - 若内层查询结果为空，则外层的WHERE子句返回真值

3.3 SQL数据查询语言

➢带有EXISTS谓词的子查询

- *例3-54：查询所有选修了1号课程的学生姓名*
- 思路分析：
  - 本查询涉及Student和SC关系
  - 在Student中依次取每个元组的Sno值，用此值去检查SC关系
  - 若SC中存在这样的元组，其Sno值等于此Student.Sno值，并且其Cno= '1'，则取此Student.Sname送入结果关系

3.3 SQL数据查询语言

➢带有EXISTS谓词的子查询

- 用嵌套查询

```
SELECT Sname
FROM   Student
WHERE EXISTS(SELECT *
        FROM    SC
        WHERE SC.Sno=Student.Sno
           AND Cno= ' 1 ');
```

3.3 SQL数据查询语言

➢集合查询

- 集合操作的种类
  - 并操作UNION
  - 交操作INTERSECT
  - 差操作EXCEPT
- 参加集合操作的各查询结果的列数必须相同；对应项的数据类型也必须相同

3.3 SQL数据查询语言

➢集合查询

- *例3-57：查询计算机科学系的学生及年龄不大于19岁的学生*
- 方法一：

```
SELECT *
FROM Student
WHERE Sdept= 'CS'
UNION
SELECT *
FROM Student
WHERE Sage<=19;
```

- UNION：将多个查询结果合并起来时，系统自动去掉重复元组。
- UNION ALL：将多个查询结果合并起来时，保留重复元组

3.3 SQL数据查询语言

➢集合查询

- *例3-57：查询计算机科学系的学生及年龄不大于19岁的学生*
- 方法二：

```
SELECT *
FROM Student
WHERE Sdept= 'CS'  OR  Sage<=19;
```

3.3 SQL数据查询语言

➢集合查询

- *例3-58：查询选修了课程1或者选修了课程2的学生*

```
SELECT Sno
FROM SC
WHERE Cno=' 1 '
UNION
SELECT Sno
FROM SC
WHERE Cno= ' 2 ';
```

3.3 SQL数据查询语言

➢集合查询

- *例3-59：查询计算机系(CS)的学生与年龄不大于19岁的学生的交集*

```
SELECT *
FROM STUDENT
WHERE Sdept='CS'
INTERSECT
SELECT *
FROM STUDENT
WHERE Sage<=19
```

3.3 SQL数据查询语言

➢集合查询

- 实际上就是查询计算机系(CS)中年龄不大于19岁的学生:

```
SELECT *
FROM Student
WHERE Sdept= ‘CS' AND  Sage<=19；
```

3.3 SQL数据查询语言

➢集合查询

- *例3-60：查询选修课程1的学生集合与选修课程2的学生集合的交集*

```
SELECT Sno
FROM SC
WHERE Cno=' 1 '
INTERSECT
SELECT Sno
FROM SC
WHERE Cno=' 12'
```

3.3 SQL数据查询语言

➢集合查询

- 实际上是查询既选修了课程1又选修了课程2的学生

```
SELECT Sno
FROM SC
WHERE Cno=' 1 '
AND Sno IN (SELECT Sno
        FROM SC
        WHERE Cno=' 2 ');
```

3.3 SQL数据查询语言

➢集合查询

- *例3-61：查询计算机系(CS)的学生与年龄不大于19岁的学生的差集*

```
SELECT *
FROM Student
WHERE Sdept='CS'
EXCEPT
SELECT  *
FROM Student
WHERE Sage <=19;
```

3.3 SQL数据查询语言

➢集合查询

- 实际上是查询计算机系(CS)中年龄大于19岁的学生

```
SELECT *
FROM Student
WHERE Sdept= 'CS' AND  Sage>19；
```

3.3 SQL数据查询语言

➢集合查询

- *例3-62：查询至少选修了“CS－02”或“CS－04”课程的学生学号：*

```
(SELECT S#
FROM SC
WHERE C# ＝‘CS－02’)
UNION
(SELECT S#
FROM SC
WHERE C#＝’CS－04’)
```

3.4 SQL数据操纵功能

- 数据插入
- 数据删除
- 数据修改

3.4 SQL数据操纵功能

➢数据插入

- SQL的数据插入语句用于将记录加入到数据库表中，其常用形式包括单记录插入和子查询结果插入两种
- 单记录插入的语法格式如下，INSERT是SQL用于表示记录添加的保留字

INSERT INTO <表名> [(列名，…列名)]

VALUES (列值，…列值)

3.4 SQL数据操纵功能

➢数据插入

- *例3-63：将数据库课程加入到Course表中：*

```
INSERT INTO Course (C#，Cname，Teacher)
VALUES (‘CS－06’,‘数据库系统原理’,’李华’)
```

- *例3-64：在SC表中增加记录(‘01055123’, ‘CS - 06’)，成绩暂缺：*

```
INSERT INTO SC(S#，C#)
VALUES (‘01055123’,’CS－06’)
```

3.4 SQL数据操纵功能

➢数据插入

- SQL3允许采用上述的单记录插入方式一次向表中插入多个记录，其形式如下：

  INSERT INTO <表名> [(列名，…列名)]

  VALUES (列值，…列值)，…，(列值，…列值)

- 另一种多记录插入的方式是子查询结果插入，即将某个表中的若干个记录按照一定的查询条件作为一个查询的结果集插入到另一个表中，形式如下：

  INSERT INTO <表名> [(列名，…列名)]

  <子查询>

3.4 SQL数据操纵功能

➢数据插入

- *例3-64： 将平均成绩高于80分的男生学号及平均成绩存入表S_Grade(S#, Avg_Grade)中，其中S#表示学号，Avg_Grade表示平均成绩：*

```
INSERT INTO S_Grade(S#，Avg_Grade)
SELECT S#，AVG(Grade)
FROM SC
WHERE S# IN
      (SELECT  S#
       FROM   Student
       WHERE  SEX=’男’)
GROUP  BY S#
      HAVING  AVG(Grade) >80
```

3.4 SQL数据操纵功能

➢数据删除

- SQL的数据删除语句可以将表中的部分或全部记录清除，语法格式如下：

  DELETE　FROM　<表名>

  [WHERE 条件表达式]

- 如果该语句指定WHERE条件，则只有满足条件的那些记录才会被删除，条件可以是简单的算术比较表达式，也可以是带有子查询的复杂形式

3.4 SQL数据操纵功能

➢数据删除

- *例3-65：删除课程表Course中的全部记录：*

```
DELETE  FROM  Course
```

- *例3-66：删除Student表中学号为’01055123’的学生记录：*

```
DELETE  FROM  Student
WHERE   S#＝’01055123’
```

3.4 SQL数据操纵功能

➢数据删除

- *例3-67：将“C语言”课程成绩不存在的记录删除：*

```
DELETE FROM SC
WHERE (Grade IS NULL)
      AND C# IN
      (SELECT C#
       FROM Course
       WHERE Cname =’C语言’)
```

3.4 SQL数据操纵功能

➢数据修改

- SQL的数据修改语句用于更新表中已有记录在不同列上的取值，语法格式如下：

  UPDATE　<表名>

  SET　<列名> = <列值表达式>[，<列名> = <列值表达式>…]

  [WHERE 条件表达式]

3.4 SQL数据操纵功能

➢数据修改

- *例3-68：将雇员表EMP中每名员工的工资增加100元：*

```
UPDATE EMP
SET   Salary＝Salary＋100
```

- *例3-69：将“01055071”同学选修的“CS－03”课程成绩置为NULL：*

```
UPDATE   SC
SET    Grade＝NULL
WHERE  S#＝’01055071’
AND    C#＝’CS－03’
```

3.5 SQL视图

- SQL视图是不存储具体数据，而仅在数据目录中存放其定义的“虚表”，它提供了一种间接访问基本表中数据的便捷方式，使用户可以更有效，更安全的访问系统中存储的相关数据
- 视图可以通过基本表或其它视图导出，因而有着许多和基本表类似的性质
- 本节介绍视图的使用方法及应该注意的问题，具体包括视图的定义、删除、查询、更新，以及视图的应用特点等几方面内容

3.5 SQL视图

➢视图的定义

- 可以通过以下方式建立视图：

  CREATE VIEW <视图名>[(<列名>，…，<列名>)]

  AS <子查询>

  [WITH CHECK OPTION]

- 视图名称后指定的列名表部分不是必须的，如果不给出这些具体的列名，则视图的各列与子查询中SELECT语句所指定的列完全相同；
- 如果在子查询的SELECT语句中使用了函数或运算表达式，那么必须在视图名后给出全部的列名表，因为函数和运算表达式是不能用来表示一个列的名称的

3.5 SQL视图

➢视图的定义

- *例3-70：建立全部男同学的视图：*

```
CREATE VIEW S_MALE
AS  SELECT S#，Class，Sname
      FROM  Student
      WHERE  SEX=’男’
WITH CHECK OPTION
```

3.5 SQL视图

➢视图的定义

- *例3-71：建立一个计算机系学生学号、选修课程数及其平均成绩的视图：*

```
CREATE VIEW S_AG (S#，CNT_C，AVG_G)
AS SELECT S#，COUNT(C#)，AVG(Grade)
FROM  SC
      WHERE  C# LIKE ‘CS%’
     GROUP  BY S#
```

3.5 SQL视图

➢视图的定义

- *例3-72：建立选修了“操作系统”课程的学生学号、姓名、及成绩的视图：*

```
CREATE VIEW S_C
AS SEELCT S.S#，Sname，Grade
      FROM Student，Course，SC
      WHERE Student.S#＝SC.S# AND Course.C#＝
      SC.C#
         AND Cname＝’操作系统’
```

3.5 SQL视图

➢视图的删除

- 删除视图的语句如下：

```
DROP VIEW <视图名>
```

- *例3-73：删除视图S_MALE：*

```
DROP VIEW S_MALE
```

- 注意：DROP TABLE XX中，基本表定义一旦删除，建立在此表上的视图虽仍然保留，但已无法引用，因此执行删除操作一定要格外小心

3.5 SQL视图

➢视图的查询

- 由于视图中并不存储数据记录，因此处理视图查询时系统会首先将视图查询语句中的WHERE条件与视图定义中的WHERE条件进行有效合并，然后再转变为对基本表的查询

- *例3-74：在视图S_MALE上查询“李华”同学的学号及所在班级：*

```
SELECT S#，Class
FROM  S_MALE
WHERE  Sname＝’李华’
```

3.5 SQL视图

➢视图的查询

- 视图S_MALE建立在基本表Student上，因此本查询也将转换为如下所示的基本表Student上的查询：

```
SELECT S#，Class
FROM  Student
WHERE Sname＝’李华’ AND SEX＝’男’
```

- *例3-75：在视图S_AG中查询学号为’01055107’的计算机系同学的选课情况：*

```
SELECT CNT_C，AVG_G
FROM  S_AG
WHERE S#＝’01055107’
```

3.5 SQL视图

➢视图的更新

- 视图的更新（包括INSERT、UPDATE、DELETE等操作）往往是一件非常复杂的事情。我们知道视图的更新最终还是会落实为对基本表的更新，但并不是所有的视图记录都能够惟一的对应到基本表中的一条记录，因此视图的更新通常会具有较大的限制
- *例3-76：在视图S_MALE中将学号为‘01055029’的学生所在班级更新为‘计算机12’：*

```
UPDATE S_MALE
SET    Class＝’计算机12’
WHERE  S#＝’01055029’
```

3.5 SQL视图

➢视图的更新

- *例3-77：在视图S_MALE中插入记录('01055072'，'计算机13'，'王晓斌')：*

```
INSERT INTO S_MALE
VALUES ('01055072', '计算机13', '王晓斌')
```

- *该操作同样也可以正常执行，系统会将其转换为如下的基本表操作：*

```
INSERT INTO Student
VALUES ('01055072', '计算机13', '王晓斌', '男')
```

3.5 SQL视图

➢视图的更新

- *例3-78：将视图S_AG中学号为 ‘01055016’的同学的选课数更新为12，并将其平均成绩更新为81.7分：*

```
UPDATE S_AG
SET   CNT_C＝12，AVG_G＝81.7
WHERE  S#＝‘01055016’
```

- *例3-79：在视图S_C中插入新记录(‘01055097’，‘张明’，92)：*

```
INSERT INTO S_C
VALUES (‘01055097’，‘张明’，92)
```

3.5 SQL视图

➢视图的更新

- 总结出视图更新的特点及限制如下：

（1）行列子集视图是可以更新的。一个例外情况是，如果基本表并没有为视图定义中不包含的那些属性列指定缺省值，同时这些列又不允许取值为NULL，那么这样的行列子集视图是不能执行INSERT操作的

（2）如果视图定义中使用了DISTINCT、GROUP BY分组或者是聚集函数，那么这样的视图是不能更新的

（3）如果视图定义中使用了多表联接或者含有嵌套查询，那么通常情况下这样的视图是不能更新的

3.5 SQL视图

➢视图的应用

- 视图对应于数据库三层模式结构的外模式，视图这个概念的提出极大的方便了关系数据库系统的使用，具体的说，视图应用具有以下的几个优点：

（1）提供了数据的逻辑独立性

（2）简化了用户应用

（3）提供了一定的数据安全保护功能

3.6 SQL数据控制功能

- 由DBMS提供统一的数据控制功能是数据库系统的特点之一
- 数据控制亦称为数据保护，包括数据的安全性控制、完整性控制、并发控制和恢复
- 这里主要介绍SQL的数据控制功能

3.6 SQL数据控制功能

➢权限与角色

- 数据库上的权限主要包括两种，一种是用户级的权限，另一种是表级的权限
- 用户级权限是指DBA可以单独授予某个用户在数据库上可进行何种操作的权限
- SQL标准支持的是表级权限，这是指DBA可以单独控制每个基本表和视图的存取，可能会经常用到的表级权限包括：表（包括基本表和视图）的查询(SEELCT)、更新(INSERT、UPDATE、DELETE)和引用(REFERENCE)等

3.6 SQL数据控制功能

➢权限与角色

- 除了可以给一个特定的用户授予权限外，SQL还支持角色的概念
- 角色并不对应于某个具体的用户，而是对于一类具有共同特征的用户的总称，这样作的目的是为了便于管理
- SQL中定义角色的语法如下：

  CREATE ROLE <角色名>

  [WITH ADMIN {<当前用户名>|<当前角色名>}]

3.6 SQL数据控制功能

➢权限的授予和收回

- 将表级权限授予某个授权ID的语法格式如下：

  GRANT <权限> ON TABLE <表名>

  TO <授权ID>，…，<授权ID> [WITH GRANT OPTION]

- 其中权限包括SELECT、UPDATE、INSERT、DELETE、REFERENCE、ALL PRIVILEGES等，ALL PRIVILEGES表示授予所有的权限，此外对于SELECT和UPDATE还可以将权限指定到具体的某些属性列上
- 如果使用了WITH GRANT OPTION，则表明该权限是可以转授的

3.6 SQL数据控制功能

➢权限的授予和收回

- *例3-80：将表SC上的SELECT和UPDATE(Grade)特权授予用户“Wang”，同时允许该用户将权限授予其它用户:*

```
GRANT SELECT，UPDATE(Grade) ON TABLE SC
TO Wang WITH GRANT OPTION
```

- 收回用户某种特权的SQL语句如下:

```
REVOKE <权限> ON TABLE <表名>
FROM <授权ID>，...，<授权ID>
```

- *例3-81：将用户Wang在SC表上的UPDATE权限收回:*

```
REVOKE UPDATE(Grade) ON TABLE SC FROM Wang
```

3.7 嵌入式SQL

➢SQL语言提供了两种使用方式：

- 一种是在终端交互式方式下使用，前面介绍的就是做为独立语言由用户在交互环境下使用的SQL语言
- 另一种是将SQL语言嵌入到某种高级语言如PL/1、COBOL、FORTRAN、C中使用，利用高级语言的过程性结构来弥补SQL语言在实现复杂应用方面的不足，这种方式下使用的SQL语言称为嵌入式SQL（Embedded SQL），而嵌入SQL的高级语言称为主语言或宿主语言

3.7 嵌入式SQL

- 在嵌入式SQL中，为了能够区分SQL语句与主语言语句，所有SQL语句都必须加前缀

  EXEC SQL

- SQL语句的结束标志则随主语言的不同而不同，例如在PL/I和C中以分号（;）结束，

  在COBOL中以END-EXEC结束

- 这样，以C或PL/I作为主语言的嵌入式SQL语句的一般形式为：

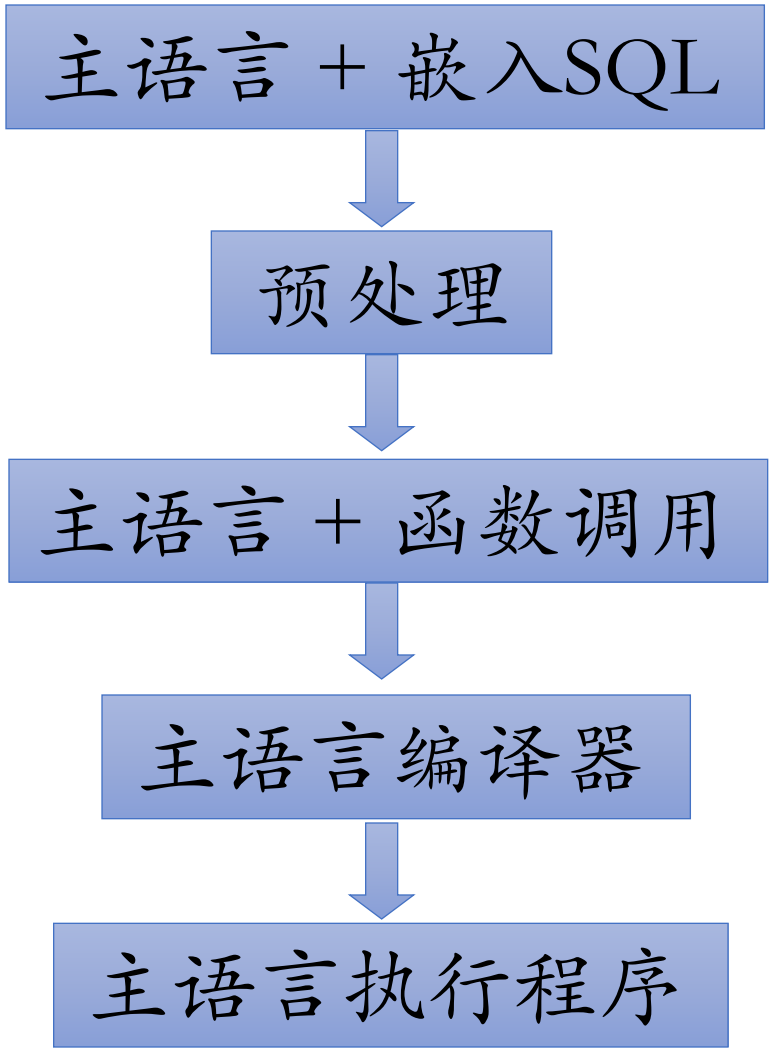
  EXEC SQL <SQL语句>;

3.7 嵌入式SQL

- 嵌入式SQL语句根据其作用的不同，可分为可执行语句和说明性语句两类
- 可执行语句又分为：数据定义、数据控制、数据操纵三种
- 在宿主程序中，任何允许出现可执行的高级语言语句的地方，都可以写可执行SQL语句
- 任何允许出现说明性高级语言语句的地方，都可以写说明性SQL语句

3.7 嵌入式SQL

➢嵌入式SQL的运行过程



3.7 嵌入式SQL

➢嵌入式SQL与主语言之间的通信

- 数据库工作单元与源程序工作单元之间通信主要包括：

1. 向主语言程序传递SQL语句的执行状态信息，使主语言能够据此控制程序流程

2. 主语言程序向SQL语句提供参数

3. 将SQL语句查询数据库的结果交主语言程序进一步处理

3.7 嵌入式SQL

- 嵌入式SQL（Embedded SQL，ESQL）是将SQL语句嵌入到程序设计语言中
- 被嵌入的程序设计语言称为宿主语言，如C、C++、Java、Python等
- 通过调用宿主语言中可以用来访问数据库的包或库，可以达到使用嵌入式SQL来访问数据库的目的

- 在使用嵌入式SQL访问数据库的步骤：
  - 建立与数据库的连接，以验证访问的合法性
  - 使用嵌入式SQL来访问数据库
  - 释放所占用的连接资源，关闭程序与数据库的连接

3.7 嵌入式SQL

- 嵌入式SQL连接数据库：EXEC SQL CONNECT TO 数据库名字AS连接名字 USER用户名;
- 嵌入式SQL断开连接：EXEC SQL DISCONNECT 连接名字或EXEC SQL DISCONNECT CURRENT;
- 嵌入式SQL提交与撤销：
  - 执行提交语法为 EXEC SQL COMMIT WORK;
  - 执行撤消语法为 EXEC SQL ROOLBACK WORK;
  - 直接提交或者撤销后断开连接，即EXEC SQL COMMIT RELEASE和 EXEC SQL ROOLBACK RELEASE。
- 嵌入式SQL查询：嵌入式SQL可以将SQL查询结果赋值给高级语言的变量。
  - EXEC SQL BEGIN DECLARE SECTION; //开始声明
  - char cSname[10], queryName[10]=“李博”; int cSage; //声明变量
  - EXEC SQL END DECLARE SECTION; //结束声明
  - EXEC SQL SELECT Sname, Sage INTO :cSname, :cSage FROM Student WHERE Sname = :queryName ; 定义的变量可以输入给SQL（例如queryName），也可以从SQL获取值（例如cSName和cSage）。

3.7 嵌入式SQL

- 在通过嵌入式SQL访问数据库时，嵌入式SQL语句与宿主语言之间要进行数据传输。
- 高级语言一般一次处理一条元组。当SQL查询结果为多条元组时，需解决SQL和高级语言转换的问题。
- 嵌入式SQL的数据传输是通过游标来实现一次获得一个元组。
- 游标存储嵌入式SQL语句的查询结果集合，每个SQL语句会返回一个游标（即返回SQL一条结果）。游标指向某条记录集的指针，通过这个指针移动, 每次读一行数据处理一行，直到处理完毕。
- 在程序中使用宿主语言访问游标，可以获取查询的结果。游标的使用过程主要包括以下四个步骤：

  （1）定义游标：将游标与SQL相关联。EXEC SQL DECLARE curStudent CURSOR FOR SELECT Sno, Sname FROM Student WHERE Sno='1' ;

  （2）打开游标：执行与游标相关联的SQL，查询结果集合会被存储到游标的数据缓冲区。例如EXEC SQL OPEN curStudent;

  （3）使用游标：从游标中读取数据（即查询结果集合），每次读取一条记录，也可以一次性读取全部记录。EXEC SQL FETCH curStudent INTO :cSno, :cSname; 将读取的一条记录的属性值赋值给两个变量cSno和cSname。

  （4）关闭游标：停止使用游标，释放游标的数据缓冲区。例如EXEC SQL CLOSE curStudent

3.7 嵌入式SQL

- 嵌入式SQL语句与宿主语言之间的数据传输是通过游标实现的。
- 游标存储嵌入式SQL语句的查询结果集合，每个SQL查询语句会返回一个游标（即返回SQL查询的结果）。
- 在程序中使用宿主语言访问游标，可以获取查询的结果。
- 游标的使用过程主要包括以下四个步骤：

1. 定义游标：将游标与SQL查询语句相关联。
2. 打开游标：执行与游标相关联的SQL，查询结果集合会被存储到游标的数据缓冲区。
3. 使用游标：从游标中读取数据，可以每次读取一条记录，也可以一次性读取全部记录。
4. 关闭游标：停止使用游标，释放游标的数据缓冲区。

3.7 嵌入式SQL

➢动态SQL

- 前面节中介绍的嵌入式SQL语句为编程提供了一定的灵活性，使用户可以在程序运行过程中根据实际需要输入WHERE子句或HAVING子句中某些变量的值。
- 这些SQL语句的共同特点是，语句中主变量的个数与数据类型在预编译时都是确定的，只有主变量的值是程序运行过程中动态输入的，我们称这类嵌入式SQL语句为静态SQL语句。

3.7 嵌入式SQL

➢动态SQL

- 静态SQL语句提供的编程灵活性在许多情况下仍显得不足，有时候我们需要编写更为通用的程序
- 例如，查询学生选课关系SC，任课教师想查选修某门课程的所有学生的学号及其成绩，班主任想查某个学生选修的所有课程的课程号及相应成绩，学生想查某个学生选修某门课程的成绩，也就是说查询条件是不确定的，要查询的属性列也是不确定的，这时就无法用一条静态SQL语句实现了

3.7 嵌入式SQL

➢动态SQL

- 实际上，如果在预编译时下列信息:
  - 主变量个数
  - 主变量的数据类型
  - SQL语句中引用的数据库对象(例如列、索引、基本表、视图等)
  - SQL语句正文
- 不能确定，我们就必须使用动态SQL技术

3.7 嵌入式SQL

➢动态SQL

- 动态SQL方法允许在程序运行过程中临时“组装”SQL语句，主要有三种形式：

1) 语句可变

2) 条件可变

3) 数据库对象、查询条件均可变

3.7 嵌入式SQL

➢动态SQL

- *例3-88: 下面两个C语言的程序段说明了动态SQL语句的使用技术*

①

```
EXEC SQL BEGIN DECLARE SECTION;
    char *query;
        EXEC SQL END DECLARE SECTION;
        scanf（”%s”，query）；/* 从键盘输入一个SQL语句 */
        EXEC SQL PREPARE que FROM： query;
        EXEC SQL EXECUTE que;
```

- 这个程序段表示从键盘输入一个SQL语句到字符数组中；字符指针query指向字符串的第1个字符，如果执行语句只做一次，那么程序段最后两个语句可合并成一个语句：

```
EXEC SQL EXECUTE IMMEDIATE： query;
```

3.7 嵌入式SQL

➢动态SQL

②

```
char *query =″ UPDATE sc SET grade = grade * 1.1
            WHERE c# = ? ″ ;
  char cno[5] = ″ C4″ ;
  EXEC SQL PREPARE dynprog FROM :query;
  EXEC SQL EXECUTE dynprog USING :cno;
```

- 这个程序段表示从这里第一个char语句表示用户组合成一个SQL语句，但有一个值（课程号）还不能确定，因此用“？”表示
- 第二个语句（char语句）表示取到了课程号值
- 第三个语句是动态SQL预备语句
- 第四个语句是动态SQL执行语句，“？”值到共享变量cno中取

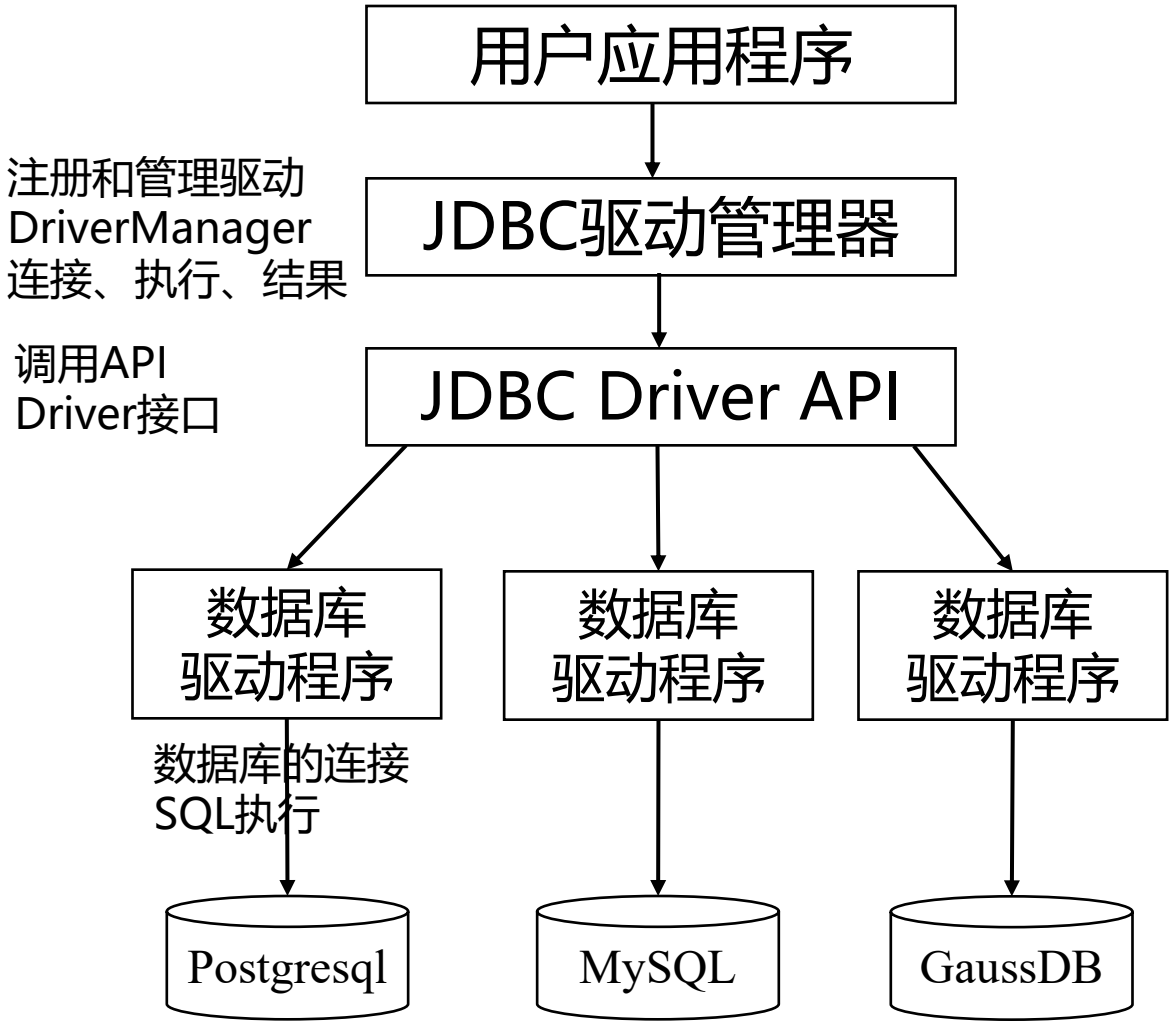
3.7 嵌入式SQL

➢JDBC

- JDBC（Java Database Connectivity，Java数据库连接）是一种应用程序编程接口，是Java开发工具包（Java Development Kit，JDK）的一部分。
- JDBC提供Java程序访问数据库的标准接口，Java应用程序可以基于JDBC访问不同类型的数据库。
- JDBC依赖于两个软件组件：
  - java.sql包：JDBC访问数据库的标准接口
  - JDBC驱动程序：实现java.sql中定义的访问数据库的接口，从而从数据库中获取数据
- 通过编写调用java.sql中接口的代码，Java程序员可以实现对数据库的访问。

3.7 嵌入式SQL

➢JDBC



3.7 嵌入式SQL

➢**JDBC**

```
Connection conn=null;//初始化连接
Statement stmt=null;//初始化执行对象，提交SQL
ResultSet rs=null;//SQL结果集
Class.forName(com.DB.jdbc.Driver);  //加载JDBC的数据库驱动
conn=DriverManager.getConnection("localhost", "root", "123456");  //连接数据库
stmt=conn.createStatement();  //实例化Statement对象
ResultSet rs=stmt.executeQuery("SELECT * FROM Student WHERE Sage>=18"); //执行SQL查询
while (rs.next()) {  //获取SQL查询结果中的一条记录
  String sno=rs.getString("Sno"); //获取SQL查询结果中的一条记录的各个字段
  String sname=rs.getString("Sname");
}
rs.close();  //关闭查询结果
stmt.close();  //关闭Statement对象
conn.close();  //关闭连接
```

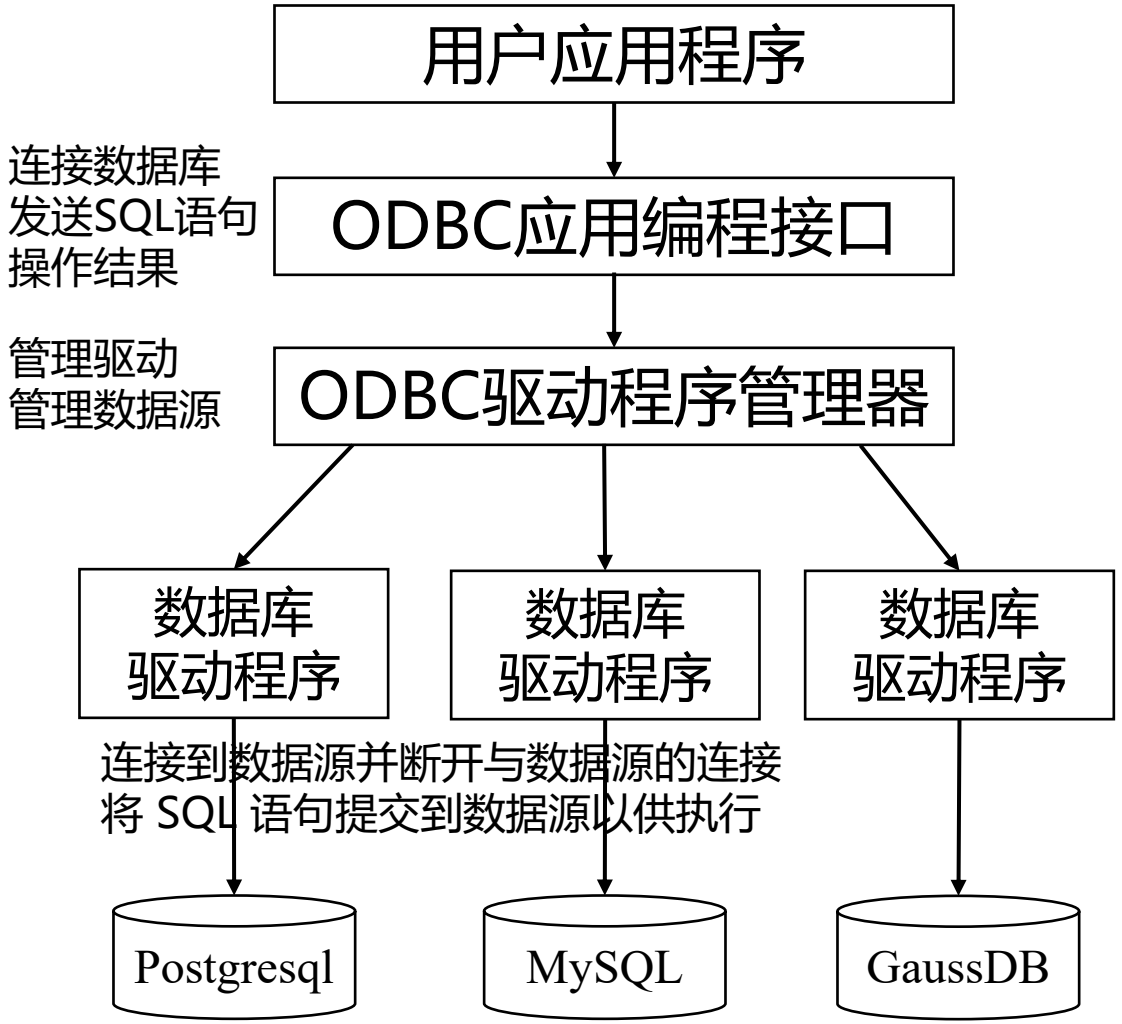
3.7 嵌入式SQL

➢ODBC

- ODBC（Open Database Connectivity，开放数据库连接）是微软公司建立的一组访问数据库的标准应用程序接口。
- ODBC可以用于任何程序设计语言
- 对于不同的数据库系统，ODBC有统一的访问数据库的接口，并为不同的数据库系统实现了相应的ODBC驱动程序。
- 当ODBC应用程序访问数据库时，其访问请求会被驱动程序管理器提交给相应的数据库系统的ODBC驱动程序，然后ODBC驱动程序会返回对数据库的结果。

3.7 嵌入式SQL

➢ODBC



3.7 嵌入式SQL

➢ODBC

```
SQLHENV henv; //定义环境句柄：设置当前程序的整体属性，如ODBC的版本，最大连接数等
SQLHDBC hdbc; //定义连接句柄：控制当前连接的属性，如事务自动提交、连接类型、隔离级别等
SQLHSTMT hstmt; //定义语句句柄：完成数据库对象及数据的操作
SQLRETURN ret=SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &henv); //分配环境句柄
ret=SQLSetEnvAttr(henv, SQL_ATTR_ODBC_VERSION, (void*)SQL_OV_ODBC3, 0);//设置ODBC环境属性
ret=SQLAllocHandle(SQL_HANDLE_DBC, henv, &hdbc); //分配连接句柄
ret=SQLConnect(hdbc, ”DB", SQL_NTS, "root", SQL_NTS, "123456", SQL_NTS);
if(SQL_SUCCEEDED(ret)){
 ret=SQLAllocHandle(SQL_HANDLE_STMT, hdbc, &hstmt); //分配语句句柄
 ret=SQLExecDirect(hstmt, "SELECT * FROM Student WHERE Sage>=18", SQL_NTS);//执行SQL查询
 SQLCHAR sno[10]; SQLCHAR sname[10];
 while(SQL_SUCCEEDED(ret=SQLFetch(hstmt))) //获取SQL查询结果中的一条记录{
     //获取SQL查询结果中的一条记录的各个字段
     ret=SQLGetData(hstmt, 1, SQL_C_CHAR, sno, 10, NULL);
     ret=SQLGetData(hstmt, 2, SQL_C_CHAR, sname, 10, NULL); }}
SQLFreeHandle(SQL_HANDLE_STMT, hstmt);  //释放语句句柄
SQLDisconnect(hdbc);                    //关闭连接
SQLFreeHandle(SQL_HANDLE_DBC, hdbc);    //释放连接句柄
SQLFreeHandle(SQL_HANDLE_ENV, henv);    //释放环境句柄
```